

Bookmark Title	Page Number
CS100 Lecture 2	1
Contents > Contents	1
Variable declaration	1
Type of a variable > Type of a variable	1
Type of a variable > Statically-typed vs dynamically-typed	1
Type of a variable > Declare a variable	1
Type of a variable > Readability matters	1
Type of a variable > Use of global variables	2
Type of a variable > Initialize a variable	2
Arithmetic types	2
Integer types > Integer types	2
Integer types > Bits and bytes	2
Integer types > Integer types	3
Integer types > Width of integer types	3
Integer types > Implementation-defined behaviors	3
Integer types > Which one should I use?	3
Integer types > Real floating types	3
Integer types > Which one should I use?	3
Integer types > scanf/printf	4
Integer types > Character types	4
Integer types > ASCII (American Standard Code for Information Interchang	4
Integer types > [Best practice] Avoid magic numbers	5
Integer types > Escape sequence	5
Integer types > Character types	6
Integer types > Boolean type: bool (since C99)	6
CS100 Lecture 3	7
Contents > Contents	7
Operators	7
The calculator > The calculator	7
The calculator > +, -, *, /, %	7
The calculator > Binary +, - and *, /	7
The calculator > Division: a / b	8
The calculator > Remainder: a % b	8
The calculator > Compound assignment operators	9
The calculator > Signed integer overflow	9
The calculator > Undefined behavior	9
The calculator > Increment/decrement operators	9
CS100 Lecture 4	10
Contents > Contents	10
Operators	10
Operator precedence > Operator precedence	10
Operator precedence > Associativity	10
Operator precedence > Evaluation order	10
Operator precedence > Evaluation order and undefined behavior	10
Operator precedence > Terminology: Return type/value of an operator	11
Operator precedence > Comparison operators	11
Operator precedence > Logical operators	11
Operator precedence > Avoid abuse of parentheses	12
Operator precedence > Short-circuit evaluation	12
Operator precedence > Conditional operator	12
Operator precedence > Conditional operator ?:	13
Operator precedence > Assignment operator =	13

Control Flow	13
do-while > do-while	13
do-while > switch-case	14
Functions	14
Call and return > Call and return	14
Call and return > Function declaration and definition	16
Call and return > Scopes	16
Call and return > Name lookup	16
Call and return > Scopes and name lookup	17
CS100 Lecture 5	17
Contents > Contents	17
Contents > Local static variables	17
Contents > Initialization	17
Contents > Implicit initialization [Very important]	18
Contents > Uninitialized garbage can be deadly!	18
Contents > const type qualifier	18
Pointers	18
Pointers > Pointers	18
Pointers > Declare a pointer	19
Pointers > & and *	19
Pointers > *	19
Pointers > The null pointer	19
Pointers > Implicit initialization of pointers	19
Pointers > Pointers that are not dereferenceable	20
Pointers > Argument passing	20
Arrays	20
Arrays > Arrays	20
Arrays > Nested arrays	21
Arrays > Initialization of nested arrays	21
CS100 Lecture 6	22
Contents > Contents	22
Pointers and Arrays	22
Pointer arithmetic > Pointer arithmetic	22
Pointer arithmetic > Array-to-pointer conversion	22
Pointer arithmetic > Subtraction of pointers	22
Pointer arithmetic > Pointer arithmetic	22
Pointer arithmetic > Pass an array to a function	23
Pointer arithmetic > Pass an array to a function	23
Pointer arithmetic > Pass an array to a function	23
Pointer arithmetic > Subscript on pointers	23
Pointer arithmetic > Return an array?	24
Pointer arithmetic > Return an array?	24
Pointer arithmetic > Pointer type (revisited)	24
Pointer arithmetic > Pass a nested array to a function	24
Pointer arithmetic > Pointer to array	25
Pointer arithmetic > Pass a nested array to a function	25
CS100 Lecture 7	25
Contents > Contents	25
Pointers and Arrays	25
Pointer to const > Pointer to const	25
Pointer to const > const is a "lock"	26
Pointer to const > const can be helpful	26
Pointer to const > Top-level constness	26

Pointer to const > void *	26
Dynamic memory	27
A "dynamic array" > A "dynamic array"	27
A "dynamic array" > Stack memory vs heap (dynamic) memory	27
A "dynamic array" > Use malloc and free	27
A "dynamic array" > Use malloc and free	27
A "dynamic array" > Use malloc and free	27
A "dynamic array" > Use calloc	28
A "dynamic array" > malloc, calloc and free	28
A "dynamic array" > Arrays vs malloc	28
A "dynamic array" > Summary	29
A "dynamic array" > Summay	29
Strings	29
C-style strings > C-style strings	29
C-style strings > The null character '\0'	29
C-style strings > Empty string	29
C-style strings > String I/O	30
C-style strings > String I/O	30
C-style strings > String I/O	30
C-style strings > String I/O	30
C-style strings > String manipulation / examination	30
C-style strings > String manipulation / examination	30
C-style strings > String literals	31
C-style strings > Array of strings	31
CS100 Lecture 8	31
Contents > Contents	31
Command line arguments	31
Command line arguments > Command line arguments	31
Command line arguments > A new signature of main	32
Command line arguments > Command line arguments	32
Example: Read a string of unknown length	32
Read a string > Read a string	32
CS100 Lecture 9	33
Contents > Contents	33
struct	33
struct type > struct type	33
struct type > Members of a struct	33
struct type > Dynamic allocation	33
struct type > Size of a struct	34
struct type > Size of struct	34
struct type > Implicit initialization	34
struct type > Implicit initialization	34
struct type > Explicit initialization	34
struct type > Compound literals	35
struct type > struct-typed parameters	35
struct type > struct-typed parameters	35
struct type > Return a struct-typed object	35
struct type > Array member	36
struct type > Summary	36
Recursion	36
Problem 1. Calculate $n!$ > Problem 1. Calculate $n!$	36
Problem 1. Calculate $n!$ > Print a non-negative integer	36
Problem 1. Calculate $n!$ > Simplify the code	37

Problem 1. Calculate $n!$ > Print a non-negative integer	37
Problem 1. Calculate $n!$ > Design a recursive algorithm	37
Problem 1. Calculate $n!$ > Problem 3. Selection-sort	37
CS100 Lecture 11	38
IOStream: Input and Output Stream > IOStream: Input and Output Stream	38
IOStream: Input and Output Stream > Standard library file names	38
IOStream: Input and Output Stream > Namespace std	38
std::string	38
Define and initialize a string > Define and initialize a string	38
Define and initialize a string > Strings	38
Define and initialize a string > Length of a string	38
Define and initialize a string > Member function s.size() Member function	38
Define and initialize a string > Use +=	38
Define and initialize a string > Traversing a string: Use range-based for loop	39
Define and initialize a string > String IO	39
CS100 Lecture 12	39
Contents > Contents	39
References	39
Declare a reference > Declare a reference	39
Declare a reference > Declare a reference	39
Declare a reference > A reference is an alias	39
Declare a reference > A reference is an alias	40
Declare a reference > A reference must be initialized	40
Declare a reference > References must be bound to existing objects ("lvalue")	40
Declare a reference > References are not objects	40
Declare a reference > References are not objects	41
Declare a reference > References are not objects	41
Declare a reference > References are not objects	41
Declare a reference > Reference declaration	41
Declare a reference > * and &	41
Declare a reference > Example: Use references in range-for	42
Declare a reference > Example: Use references in range-for	42
Declare a reference > Example: Use references in range-for	42
Declare a reference > Example: Pass by reference-to-const	42
Declare a reference > Example: Pass by reference-to-const	43
Declare a reference > Example: Pass by reference-to-const	43
Declare a reference > Example: Pass by reference-to-const	43
Declare a reference > Example: Pass by reference-to-const	43
Declare a reference > Example: Pass by reference-to-const	44
Declare a reference > Example: Pass by reference-to-const	44
Declare a reference > Example: Pass by reference-to-const	44
Declare a reference > Benefits of passing by reference-to-const	45
Declare a reference > Benefits of passing by reference-to-const	45
Declare a reference > References vs pointers	45
std::vector	45
Class template > Class template	45
Class template > Create a std::vector	46
Class template > Create a std::vector	46
Class template > Create a std::vector	46
Class template > Create a std::vector	46
Class template > C++17 CTAD	47
Class template > Size of a std::vector	47
Class template > Append an element to the end of a std::vector	47

Class template > Remove the last element of a std::vector	47
Class template > Remove the last element of a std::vector	47
Class template > v.back() and v.front()	47
Class template > Range-based for loops	48
Class template > Range-based for loops	48
Class template > Access through subscripts	48
Class template > Feel the style of STL	48
CS100 Lecture 13	48
Contents > Contents	48
Contents > "Better C"	49
Contents > "Better C"	49
Contents > "Better C"	49
Type System	49
Stronger type checking > Stronger type checking	49
Stronger type checking > Stronger type checking	49
Stronger type checking > Stronger type checking	50
Stronger type checking > Explicit Casts	50
Stronger type checking > const_cast	50
Stronger type checking > reinterpret_cast	50
Stronger type checking > static_cast	50
Stronger type checking > Minimize casting	51
Stronger type checking > Type deduction	51
Stronger type checking > auto	51
Stronger type checking > auto	51
Stronger type checking > auto	51
Stronger type checking > auto	51
Stronger type checking > auto	52
Stronger type checking > auto	52
Stronger type checking > decltype	52
Stronger type checking > Note on auto and decltype	52
Functions	53
Default arguments > Default arguments	53
Default arguments > Default arguments	53
Default arguments > Function overloading	53
Default arguments > Overloaded functions	53
Default arguments > Overloaded functions	54
Default arguments > Overloaded functions	54
Default arguments > Basic overload resolution	54
Default arguments > Basic overload resolution	54
Default arguments > Basic overload resolution	55
Default arguments > Basic overload resolution	55
Default arguments > The null pointer	55
Default arguments > Better null pointer: nullptr	56
Default arguments > Avoid abuse of function overloading	56
Range-based for loops revisited	56
Range-based for loops > Range-based for loops	56
Range-based for loops > Range-based for loops	57
Range-based for loops > Traverse an array	57
Range-based for loops > Pass an array by reference	57
Range-based for loops > Pass an array by reference	57
Range-based for loops > Pass an array by reference	58
CS100 Lecture 14	58
Contents > Contents	58

Members of a class	58
A simple class > A simple class	58
A simple class > Member access	58
A simple class > Access control	59
A simple class > Access control	59
A simple class > Access control	59
A simple class > The this pointer	59
A simple class > The this pointer	60
A simple class > The this pointer	60
A simple class > The this pointer	60
A simple class > const member functions	60
A simple class > const member functions and the this pointer	61
A simple class > const member functions	61
A simple class > const member functions	61
A simple class > const member functions	61
Constructors	62
Constructors > Constructors	62
Constructors > Constructors	62
Constructors > Constructor initializer list	62
Constructors > Order of initialization	63
Constructors > Constructor initializer list	63
Constructors > In-class initializers	63
Constructors > Constructor initializer list	63
Constructors > Constructor initializer list	63
Constructors > Constructor initializer list	64
Constructors > Constructor initializer list	64
Constructors > Default constructors	64
Constructors > Default Constructors	64
Constructors > Default constructors	64
Constructors > Is a default constructor needed?	65
Constructors > Is a default constructor needed?	65
Constructors > Is a default constructor needed?	65
Constructors > Is a default constructor needed?	65
Constructors > Is a default constructor needed?	66
Constructors > Is a default constructor needed?	66
Constructors > Summary	66
Constructors > Summary	66
Constructors > Summary	66
CS100 Lecture 15	66
Contents > Contents	66
Constructors and destructors	66
Lifetime of an object > Lifetime of an object	66
Lifetime of an object > Lifetime of an object	67
Lifetime of an object > Constructors and Destructors	67
Lifetime of an object > Constructors and Destructors	67
Lifetime of an object > Constructors and Destructors	67
Lifetime of an object > Destructor	68
Lifetime of an object > Destructor	68
Lifetime of an object > Example: A dynamic array	68
Lifetime of an object > Dynarray: members	68
Lifetime of an object > Dynarray: constructors	68
Lifetime of an object > Dynarray: constructors	69
Lifetime of an object > Dynarray: constructors	69

Lifetime of an object > Dynarray: destructor	69
Lifetime of an object > Dynarray: destructor	69
Lifetime of an object > Dynarray: destructor	69
Lifetime of an object > Dynarray: some member functions	70
Lifetime of an object > Dynarray: some member functions	70
Lifetime of an object > Dynarray: Usage	70
Copy control	71
Copy-initialization > Copy-initialization	71
Copy-initialization > Copy-initialization	71
Copy-initialization > Copy-initialization	71
Copy-initialization > Copy constructor	71
Copy-initialization > Copy constructor	72
Copy-initialization > Dynarray: copy constructor	72
Copy-initialization > Dynarray: copy constructor	72
Copy-initialization > Synthesized copy constructor	72
Copy-initialization > Defaulted copy constructor	72
Copy-initialization > Deleted copy constructor	73
Copy-initialization > Deleted copy constructor	73
Copy-initialization > Copy-assignment operator	73
Copy-initialization > Dynarray: copy-assignment operator	73
Copy-initialization > Dynarray: copy-assignment operator	74
Copy-initialization > Dynarray: copy-assignment operator	74
Copy-initialization > Dynarray: copy-assignment operator	74
Copy-initialization > Dynarray: copy-assignment operator	74
Copy-initialization > Synthesized, defaulted and deleted copy-assignment	75
Copy-initialization > [IMPORTANT] The rule of three: Reasoning	75
Copy-initialization > [IMPORTANT] The rule of three: Reasoning	75
Copy-initialization > [IMPORTANT] The rule of three: Rules	75
Copy-initialization > [IMPORTANT] The rule of three: Rules	75
Copy-initialization > [IMPORTANT] The rule of three	76
Copy-initialization > Summary	76
Copy-initialization > Summary	76
CS100 Lecture 16	76
Contents > Contents	76
Type alias members	76
Type aliases in C++: using. > Type aliases in C++: using.	76
Type aliases in C++: using. > Type alias members	77
Type aliases in C++: using. > Type alias members	77
Type aliases in C++: using. > Type alias members in the standard library	77
Type aliases in C++: using. > Type alias members in the standard library	77
static members	77
static data members > static data members	78
static data members > static data members	78
static data members > static data members	78
static data members > static data members: Example	78
static data members > static data members: Example	78
static data members > static data members: Example	79
static data members > static data members	79
static data members > static data members	79
static data members > static member functions	80
static data members > static member functions	80
friend	80
friend functions > friend functions	80

friend functions > friend functions	80
friend functions > friend functions	81
friend functions > friend functions	81
friend functions > friend classes	81
Definition and declaration	81
Definition and declaration > Definition and declaration	81
Definition and declaration > Class definition	82
Definition and declaration > Define a member function outside the class body	82
Definition and declaration > The :: operator	82
Definition and declaration > Class declaration and incomplete type	82
Definition and declaration > Class declaration and incomplete type	82
Destructors revisited	83
Destructors revisited > Destructors revisited	83
Destructors revisited > Destructors	83
Destructors revisited > Destructors	83
Destructors revisited > Destructors	83
Destructors revisited > Destructors	83
Destructors revisited > Destructors	83
Destructors revisited > Destructors	84
Destructors revisited > Constructors vs destructors	84
Destructors revisited > Compiler-generated destructors	84
Destructors revisited > Summary	84
Destructors revisited > Summary	85
Destructors revisited > Summary	85
CS100 Lecture 17	85
Contents > Contents	85
Contents > Motivation: Copy is slow.	85
Contents > Motivation: Copy is slow.	85
Contents > Motivation: Copy is slow.	85
Contents > Motivation: Copy is slow.	86
Contents > Distinguish between the different kinds of assignments	86
Contents > Rvalue References	86
Contents > Overload Resolution	86
Move Operations	87
Overview > Overview	87
Overview > The Move Constructor	87
Overview > The Move Constructor	87
Overview > The Move Constructor	87
Overview > The Move Assignment Operator	88
Overview > The Move Assignment Operator	88
Overview > The Move Assignment Operator	88
Overview > The Move Assignment Operator	88
Overview > Lvalues are Copied; Rvalues are Moved	89
Overview > Lvalues are Copied; Rvalues are Moved	89
Overview > Lvalues are Copied; Rvalues are Moved	89
Overview > Synthesized Move Operations	89
Overview > The Rule of Five: Idea	90
Overview > The Rule of Five: Rules	90
Overview > The Rule of Five	90
Overview > How to Invoke a Move Operation?	90
std::move	90
std::move > std::move	91
std::move > std::move	91
std::move > Use std::move	91

std::move > Use std::move	91
std::move > Use std::move	92
std::move > An rvalue reference is an lvalue.	92
std::move > Lvalues persist; Rvalues are ephemeral.	92
NRVO, Move and Copy Elision	92
Returning a Temporary (pure rvalue) > Returning a Temporary (pure rvalue)	92
Returning a Temporary (pure rvalue) > Returning a Temporary (pure rvalue)	93
Returning a Temporary (pure rvalue) > Returning a Named Object	93
Returning a Temporary (pure rvalue) > Named Return Value Optimization,	93
Returning a Temporary (pure rvalue) > Named Return Value Optimization,	93
Returning a Temporary (pure rvalue) > Summary	93
Returning a Temporary (pure rvalue) > Summary	94
CS100 Lecture 18	94
Contents > Contents	94
Ideas	94
Memory management is difficult! > Memory management is difficult!	94
Memory management is difficult! > Will you always remember to delete?	94
Memory management is difficult! > Idea: Make use of destructors.	95
Memory management is difficult! > What if WindowPtr is copied?	95
Memory management is difficult! > What if WindowPtr is copied?	95
Memory management is difficult! > Overview of smart pointers	95
Memory management is difficult! > Overview of smart pointers	96
std::unique_ptr	96
Design: Unique ownership of the object > Design: Unique ownership of the	96
Design: Unique ownership of the object > std::unique_ptr	96
Design: Unique ownership of the object > Creating a std::unique_ptr: Two c	96
Design: Unique ownership of the object > Default initialization of a std::unid	97
Design: Unique ownership of the object > std::unique_ptr: Automatic mem	97
Design: Unique ownership of the object > std::unique_ptr: Move-only	97
Design: Unique ownership of the object > std::unique_ptr: Move-only	97
Design: Unique ownership of the object > Express your intent precisely.	98
Design: Unique ownership of the object > Express your intent precisely.	98
Design: Unique ownership of the object > Express your intent precisely.	98
Design: Unique ownership of the object > Express your intent precisely.	98
Design: Unique ownership of the object > Returning a std::unique_ptr	98
Design: Unique ownership of the object > Other operations on std::unique	99
Design: Unique ownership of the object > std::unique_ptr for array type	99
Design: Unique ownership of the object > std::unique_ptr for array type	99
Design: Unique ownership of the object > std::unique_ptr for array type	99
Design: Unique ownership of the object > std::unique_ptr for array type	99
Design: Unique ownership of the object > Use standard library containers i	99
Design: Unique ownership of the object > std::unique_ptr is zero-overhead	99
std::shared_ptr	99
Motivation > Motivation	99
Motivation > Motivation	100
Motivation > Idea: Reference counting	100
Motivation > Reference counting	100
Motivation > std::shared_ptr	100
Motivation > Create a shared_ptr	100
Motivation > Operations	101
Motivation > Operations	101
Motivation > Summary	101
Motivation > Question	101

CS100 Lecture 19	102
Contents > Contents	102
Contents > Basics	102
Contents > Basics	102
Contents > Basics	102
Contents > Basics	102
Example: Rational	103
A class for rational numbers > A class for rational numbers	103
A class for rational numbers > Rational: arithmetic operators	103
A class for rational numbers > Rational: arithmetic operators	103
A class for rational numbers > [Best practice] Avoid repetition.	104
A class for rational numbers > [Best practice] Avoid repetition.	104
A class for rational numbers > Rational: arithmetic operators	105
A class for rational numbers > Rational: arithmetic and relational operators	105
A class for rational numbers > Rational: arithmetic and relational operators	105
A class for rational numbers > Rational: arithmetic and relational operators	105
A class for rational numbers > Rational: relational operators	105
A class for rational numbers > Rational: relational operators	105
A class for rational numbers > Rational: relational operators	106
A class for rational numbers > Relational operators	106
A class for rational numbers > Relational operators	106
A class for rational numbers > ++ and --	106
A class for rational numbers > ++ and --	107
A class for rational numbers > IO operators	107
A class for rational numbers > Rational: output operator	107
A class for rational numbers > Rational: input operator	107
Example: Dynarray	108
operator[] > operator[]	108
Example: WindowPtr	108
WindowPtr: indirection (dereference) operator > WindowPtr: indirection (dereference)	108
WindowPtr: indirection (dereference) operator > WindowPtr: indirection (dereference)	108
WindowPtr: indirection (dereference) operator > WindowPtr: member access	109
User-defined type conversions	109
Type conversions > Type conversions	109
Type conversions > Type conversions	109
Type conversions > Type conversions	109
Type conversions > Type conversions	110
Type conversions > Type conversion operator	110
Type conversions > Explicit type conversion	110
Type conversions > Explicit type conversion	110
Type conversions > Explicit type conversion	111
Type conversions > [Best practice] Avoid the abuse of type conversion operator	111
Type conversions > Contextual conversion to bool	111
Type conversions > Contextual conversion to bool	111
Type conversions > Summary	111
Type conversions > Summary	112
CS100 Lecture 20	112
Contents > Contents	112
Contents > Iterators	112
Contents > Iterators	112
Contents > Iterators	112
Contents > Iterators	112
Contents > Iterators	113

Contents > Iterators	113
Contents > Range-for demystified	113
Contents > Iterators: dereferenceable	113
Contents > Iterators: invalidation	113
Contents > Never use invalidated iterators or references!	114
Contents > More operations on iterators	114
Contents > More operations on iterators	114
Contents > More operations on iterators	114
Contents > Initialization from iterator range	114
Algorithms	114
Algorithms > Algorithms	115
Algorithms > Algorithms: interfaces	115
Algorithms > Algorithms: interfaces	115
Algorithms > Algorithms: interfaces	115
Algorithms > if: new syntax in C++17	115
Algorithms > Algorithms: requirements	116
Algorithms > Algorithms	116
Algorithms > Some common algorithms (<algorithm>)	116
Algorithms > Some common algorithms (<algorithm>)	116
Algorithms > Some common algorithms (<algorithm>)	116
Algorithms > Some common algorithms	117
Algorithms > Predicates	117
Algorithms > Predicates	117
Algorithms > Predicates	117
Algorithms > Predicates	118
Algorithms > Predicates	118
Algorithms > Predicates	118
Algorithms > Function objects	118
Algorithms > Function objects	118
Algorithms > Lambda expressions	118
Algorithms > Lambda expressions	119
Algorithms > Lambda expressions	119
Algorithms > Lambda expressions	119
Algorithms > Lambda expressions	119
Algorithms > Lambda expressions: capture	120
Algorithms > More on lambda expressions	120
Algorithms > Back to algorithms	120
Algorithms > Summary	120
Algorithms > Summary	120
CS100 Lecture 21	121
Contents > Contents	121
Inheritance	121
Example: An item for sale > Example: An item for sale	121
Example: An item for sale > Defining a subclass	121
Example: An item for sale > Defining a subclass	121
Example: An item for sale > protected members	121
Example: An item for sale > protected members	122
Example: An item for sale > Inheritance	122
Example: An item for sale > Inheritance	122
Example: An item for sale > Inheritance	122
Example: An item for sale > Constructor of DiscountedItem	122
Example: An item for sale > Constructor of derived classes	122
Example: An item for sale > Constructor of derived classes	123

Example: An item for sale > Constructor of derived classes	123
Example: An item for sale > Constructor of derived classes	123
Example: An item for sale > Constructor of derived classes	123
Dynamic binding	124
Upcasting > Upcasting	124
Upcasting > Upcasting: Example	124
Upcasting > Static type and dynamic type	124
Upcasting > virtual functions	124
Upcasting > virtual functions	124
Upcasting > Dynamic binding	125
Upcasting > virtual-override	125
Upcasting > virtual-override	125
Upcasting > virtual destructors	125
Upcasting > virtual destructors	125
Upcasting > virtual destructors	126
Upcasting > (Almost) completed Item and DiscountedItem	126
Upcasting > (Almost) completed Item and DiscountedItem	126
Upcasting > Usage with smart pointers	126
Upcasting > Copy-control	127
Upcasting > Synthesized copy-control members	127
Upcasting > Synthesized copy-control members	127
Upcasting > Slicing	127
Upcasting > Slicing	127
Upcasting > Downcasting	128
Upcasting > Polymorphic class	128
Upcasting > Downcasting: For polymorphic class only	128
Upcasting > dynamic_cast can be very slow	128
Upcasting > Guaranteed successful downcasting: Use static_cast. > Guarant	128
Upcasting > Avoiding dynamic_cast	128
Upcasting > Avoiding dynamic_cast	129
Upcasting > Summary	129
Upcasting > Summary	129
CS100 Lecture 22	129
Contents > Contents	130
Abstract base class	130
Shapes > Shapes	130
Shapes > Shapes	130
Shapes > Shapes	130
Shapes > Pure virtual functions	130
Shapes > Pure virtual functions	131
Shapes > Pure virtual functions	131
Shapes > Pure virtual functions and abstract classes	131
Shapes > Pure virtual functions and abstract classes	131
Shapes > Make the interface robust, not error-prone.	131
Shapes > Make the interface robust, not error-prone.	132
Shapes > Make the interface robust, not error-prone.	132
Shapes > Polymorphism	132
More on the "is-a" relationship	132
Public inheritance: The "is-a" relationship > Public inheritance: The "is-a" re	132
Public inheritance: The "is-a" relationship > Example: Every student is a per	133
Public inheritance: The "is-a" relationship > Example: Every student is a per	133
Public inheritance: The "is-a" relationship > Your intuition can mislead you.	133
Public inheritance: The "is-a" relationship > No. Not every bird can fly.	133

Public inheritance: The "is-a" relationship > No. Not every bird can fly.	134
Public inheritance: The "is-a" relationship > What about report a runtime error	134
Public inheritance: The "is-a" relationship > What about report a runtime error	134
Public inheritance: The "is-a" relationship > Example: A square is a rectangle	134
Public inheritance: The "is-a" relationship > Example: A square is a rectangle	134
Public inheritance: The "is-a" relationship > Is this really an "is-a" relationship	135
Public inheritance: The "is-a" relationship > Conclusion: Public inheritance is not	135
Inheritance of interface vs inheritance of implementation	135
Example: Airplanes for XYZ Airlines. > Example: Airplanes for XYZ Airlines.	135
Example: Airplanes for XYZ Airlines. > Example: Airplanes for XYZ Airlines.	135
Example: Airplanes for XYZ Airlines. > Impure virtual function: Interface + default	135
Example: Airplanes for XYZ Airlines. > * By defining an impure virtual function	136
Example: Airplanes for XYZ Airlines. > Separate default implementation from	136
Example: Airplanes for XYZ Airlines. > Separate default implementation from	136
Example: Airplanes for XYZ Airlines. > Separate default implementation from	136
Example: Airplanes for XYZ Airlines. > Still not satisfactory?	136
Example: Airplanes for XYZ Airlines. > Inheritance of interface vs inheritance of	136
Example: Airplanes for XYZ Airlines. > Summary	137
Example: Airplanes for XYZ Airlines. > Summary	137
Example: Airplanes for XYZ Airlines. > Summary	137
Example: Airplanes for XYZ Airlines. > Notes	137